4G — lekcje 4–5 (10:55–12:35), praca samodzielna

Instrukcja — cennik (typy i operatory) oraz stan magazynu (warunki i pętle)

Pracujesz w `htdocs/elektroserwis/`. Apache w XAMPP włączony, wszystko oglądasz przez `http://localhost/elektroserwis/...`. Kod **przepisujesz, nie kopiujesz**. Narzędzia AI: wolno zapytać o błąd, nie wolno wygenerować rozwiązania.

0. Plik z danymi: `dane-start.php`

Jeśli go nie masz z lekcji 3 — utwórz :

```
<?php
$produkty = [
    ["id" => 1,  "nazwa" => "Procesor AMD Ryzen 5 5600X",          "cena" => 899.00,
"kategoria" => "Podzespoły", "ilosc" => 4],
    ["id" => 2,  "nazwa" => "Procesor Intel Core i5-13400F",       "cena" => 949.00,
"kategoria" => "Podzespoły", "ilosc" => 2],
    ["id" => 3,  "nazwa" => "Karta graficzna RTX 4060",            "cena" => 1649.00,
"kategoria" => "Podzespoły", "ilosc" => 0],
    ["id" => 4,  "nazwa" => "Pamięć RAM Kingston Fury 16GB",       "cena" => 229.00,
"kategoria" => "Podzespoły", "ilosc" => 12],
    ["id" => 7,  "nazwa" => "Klawiatura mechaniczna Logitech G413", "cena" => 349.00,
"kategoria" => "Peryferia",  "ilosc" => 8],
    ["id" => 8,  "nazwa" => "Mysz przewodowa Razer DeathAdder",    "cena" => 199.00,
"kategoria" => "Peryferia",  "ilosc" => 15],
    ["id" => 9,  "nazwa" => "Monitor 27\" 144Hz",                  "cena" => 1199.00,
"kategoria" => "Peryferia",  "ilosc" => 1],
    ["id" => 11, "nazwa" => "Kabel HDMI 2m",                       "cena" => 29.00,
"kategoria" => "Akcesoria",  "ilosc" => 40],
];
?>
```

1. Zmienne, typy i operatory: `cennik.php` (lekcja 4)

Najpierw cztery krótkie sprawdzenia w pliku `typy.php`. Przy każdym: **zapisz w komentarzu przewidywanie, dopiero potem odśwież stronę.**

```php
<?php
// a) var_dump pokazuje TYP i wartość (echo pokazuje tylko wartość)
$nazwa = "Mysz Razer"; $cena = 199.00; $ilosc = 15; $promocja = false;
echo "<pre>"; var_dump($nazwa, $cena, $ilosc, $promocja); echo "</pre>";

// b) napisy łączy KROPKA, nie plus
$opis = "Procesor";
$opis .= " Ryzen 5";
echo "Produkt: $opis<br>";      // cudzysłów podwójny podstawia zmienną
echo 'Produkt: $opis<br>';      // pojedynczy - wypisuje dosłownie

// c) == porównuje po zamianie typów, === wymaga też tego samego typu
var_dump(5 == "5");    // ?
var_dump(5 === "5");   // ?

// d) dane z formularza są napisem; przecinek to NIE separator dziesiętny
var_dump((float) "12.50");        // ?
var_dump((float) "12,50");        // ?
var_dump(is_numeric("12,50"));    // ?
echo number_format(1234.5, 2, ',', ' ') . " zł";   // kwota po polsku
?>
```

Zasady do zapamiętania: `var_dump` mówi prawdę o typie · porównujemy `===` · napis z liczbą najpierw sprawdzamy (`is_numeric`), potem zamieniamy · `number_format` na samym końcu, przed wypisaniem.

Zadanie: `cennik.php`

- ☐ stała `const VAT = 0.23;` i zmienne: nazwa produktu, cena netto, ilość sztuk
- ☐ obliczone: cena brutto za sztukę (`$netto * (1 + VAT)`) i wartość brutto zamówienia
- ☐ wynik w tabeli HTML: nazwa, netto, brutto, ilość, wartość — wszystkie kwoty przez `number_format` z „zł"
- ☐ *rozszerzone:* rabat procentowy w zmiennej odliczony od wartości zamówienia; rata przy 10 ratach zaokrąglona do grosza (`round($x, 2)`)
- ☐ *dla chętnych:* cena netto podana jako napis `"899,00"` — zamień poprawnie (`str_replace(',', '.', ...)` , potem `is_numeric`), dopiero potem licz

Gotowe, gdy: kwoty zgadzają się z kalkulatorem, przy każdym obliczeniu jest komentarz. **Na `cennik.php` przeznacz najwyżej lekcję 4** — o 11:40 przejdź do części 2, nawet jeśli nie skończyłeś rozszerzonej.

2. Wzór dla magazynu: znane algorytmy — `algorytmy.php` (lekcja 5, ok. 15 min)

Najpierw spróbuj sam według opisu, dopiero potem zajrzyj do kodu niżej:

1. policz sumę wartości magazynu (cena × ilość dla każdego produktu)
2. policz, ile produktów ma ilość 0
3. znajdź najdroższy produkt
4. znajdź produkt o nazwie „Kabel HDMI 2m" i przerwij szukanie, gdy go znajdziesz

```php
<?php
require "dane-start.php";

$suma = 0;            // suma wartości magazynu
$brakuje = 0;         // zliczanie produktów z ilością 0
$najdrozszy = null;   // maksimum: na start "jeszcze nic"

foreach ($produkty as $p) {
    $suma += $p["cena"] * $p["ilosc"];
    if ($p["ilosc"] === 0) {
        $brakuje++;
    }
    // pierwszy produkt albo droższy od dotychczasowego
    if ($najdrozszy === null || $p["cena"] > $najdrozszy["cena"]) {
        $najdrozszy = $p;
    }
}

// wyszukiwanie liniowe: przerywamy, gdy znajdziemy
$szukane = "Kabel HDMI 2m";
$znaleziony = null;
foreach ($produkty as $p) {
    if ($p["nazwa"] === $szukane) {
        $znaleziony = $p;
        break;      // dalej nie ma po co szukać
    }
}

echo "Wartość magazynu: " . number_format($suma, 2, ',', ' ') . " zł<br>";
echo "Niedostępne: $brakuje<br>";
echo "Najdroższy: " . $najdrozszy["nazwa"] . "<br>";
echo $znaleziony === null ? "Nie znaleziono" : "Znaleziono: " . $znaleziony["nazwa"];
?>
```

Odpowiedz w komentarzu: dlaczego `$najdrozszy` zaczyna od `null`, a nie od `0`? Zmień `$szukane` na nazwę, której nie ma — co się wyświetla?

3. Samodzielnie: `magazyn.php` (lekcja 5, reszta czasu)

Podstawowe

- ☐ tabela HTML wszystkich produktów: nazwa, kategoria, cena (`number_format` + „zł"), ilość, **status**
- ☐ status przez `match(true)` z lekcji 3: ilość 0 → `brak`, poniżej 5 → `ostatnie sztuki`, w pozostałych przypadkach → `dostępny`
- ☐ pod tabelą: łączna wartość magazynu, liczba produktów niedostępnych, nazwa i cena najdroższego produktu

Rozszerzone

- ☐ kolumna z ikoną kategorii przez `match` po kategorii (z lekcji 3)
- ☐ wyszukiwanie produktu po nazwie z komunikatem „nie znaleziono"

Dla chętnych

- ☐ najtańszy produkt **dostępny** (pomija ilość 0)
- ☐ liczba produktów w każdej kategorii — trzema licznikami, bez dodatkowej tablicy

Gotowe, gdy: status zgadza się dla karty RTX (`brak`), monitora (`ostatnie sztuki`) i kabla (`dostępny`); wartość magazynu zgadza się z ręcznym przeliczeniem dla dwóch produktów; przy każdym obliczeniu jest komentarz. Na koniec zostaw `cennik.php` i `magazyn.php` otwarte w przeglądarce (dwie karty).

4. Dla szybszych: pętle `for` i `while` — `petle.php`

```
<?php
require "dane-start.php";   // "wklej tu zawartość pliku dane-start.php"

// for - gdy znamy liczbę powtórzeń
for ($i = 1; $i <= 10; $i++) {
    echo "Rata $i<br>";
}

// while - dopóki warunek jest prawdziwy
$kwota = 1000;
$miesiac = 0;
while ($kwota > 0) {
    $kwota -= 300;      // spłata 300 zł miesięcznie
    $miesiac++;
}
echo "Spłata zajmie $miesiac miesiące<br>";

// foreach - każdy element tablicy
foreach ($produkty as $p) {
    echo $p["nazwa"] . "<br>";
}
?>
```

Zanim odświeżysz stronę, zapisz w komentarzu: **ile miesięcy wyjdzie w pętli `while`?** Potem sprawdź. Następnie zmień spłatę na 250 zł i znowu przewidź wynik.

Uwaga: `while`, którego warunek nigdy nie staje się fałszywy, „zawiesza" stronę do komunikatu *Maximum execution time exceeded*. To nie awaria komputera, tylko pętla — popraw warunek.

Najczęstsze błędy: kolejność warunków (najpierw `< 5`, potem `=== 0` — „brak" nigdy się nie pokaże) · `match` bez `default` → błąd `UnhandledMatchError` · `=` zamiast `===` w `if` · brak średnika → biały ekran albo *Parse error* z numerem linii.